

# Языки программирования (Вольное слушание)

silvia.lesnaia

17 сентября 2026 г.

**09.09.26**

**Пара №2**

Функциональное программирование предполагает, что вся программа представляет из себя представление функций.

Функцию стоит редсталчть в математическом смысле.

$f : \Delta \rightarrow z$

$f(x_1, \dots, x_n) \rightarrow y$

Чистая функция - функция без побочных эффектов. В каких либо условиях функция не вызывается она дает чистый результат. Вызов функции не должен приводить к изменению окружению.

С точки зрения императивного программирования вызов функции не имеет смысла.

Программа предполагает что в ней только описание функции, программе стоит задать вопрос и он заключается в том что вызов функции зависит от определенных аргументов.

Выражение - ключевое понятие для функционального программирования. По сути функциональное программирование оперирует этим понятием.

Средства комбинирования - комбинирование средств - как сущности определенное в языке программирования можно составить в более что то единое

Средства абстракции

$\lambda$ -исчисление Черча, где все описывается функциями

## 1 Standard ML - академический язык программирования

Функция - вызов функции дает нам комбинировать сущности.

Выражение состоит из каких то данных достаточно просто

Функции задаются в функциональной форме

$+$  :  $real * real \rightarrow real$

$:$   $int * int \rightarrow int$

$abs(25 * a)$  и  $(op+)(2, 3) \Rightarrow S$  по сути разницы нет

REPL - позволяет общаться с интерпретатором  
Можно использовать как спец конструкции так и выражения  
Список конструкций:

- `val a = 25*3` - дает обобщенные - имена обозначают какое-то выражение  
`val a = abc` - теперь новое значение EEEEEEEEEЕ ЗАТЕНЕНИЕ
- `fun имя(параметры) : тип = выражение`

Затенение - оно временное или постоянное, оно может быть если используется обозначение для другого обозначения

### 10.09.26

#### Пара №3

Хвостовая рекурсия -

Лучше писать рекурсии то лучше писать их хвостовыми. В картеже объясняются значения разного типа, т.е. можно объединить значения и они могут принадлежать к разным типам.

`(25,3, 48.3,"abc")` - элементы картежа.

Типы перечисляются через `*` - `int * int * real * string`

Пример: `fun f(v : real * int) : int * real * real =`

`#1(v)` - функция вернет первый элемент кортежа

`#2(v)` - функция вернет второй элемент кортежа

`#` и число это название функции.

`(a,b)` - это объединение картежа.

Записи - это как структуры в C++, т.е. это набор имен переменных полей.

Определение: `val s = name = "Sergei" surname = "Mironov" number = 345, sum = 34.98`

Имя поля и его значение. Тип записи это тип сложный который является пересечением имен полей, с типами этих полей заключенный в фигурные скобки, причем порядок следования этих полей безразличен.

`#s.name` вернет `Sergei` и т.д.

`list` - список. Список бывает элементов определенного типа. Нужно написать тип списка - `int list` - это список нормальных чисел.

`a (int * real) list` это будет по сути чередование

`int * real list` получится список где одна часть нормальных чисел а другая вещественная

### ОООО ДАААААА СИНТАКСИЧЕСКАЯ САХАР

Это когда вводится конструкция являющаяся упрощением записи другой более сложной синтаксической конструкции.

`hd l ⇒ 2` - выдаст голову списка

`hd []` - выдаст ошибку ведь нельзя вывести голову пустого списка

`tl l ⇒ [3,6,8,15]` - хвост списка

`tl[5] ⇒ []` - вернет пустой список ведь 5:[]

`tl[]` - выдаст ошибку

`null l ⇒ false` - это функция предикат, так как возвращает истину или ложь

option - работает только с чемто, как самостоятельного его нет. Это стандартный тип данных в языках программирования. Оно выдает есть либо нет типа.

У него есть выводы: NONE - ничего нет или SOME что то есть

SOME 5 - типа int

is SOME V - выдаст true если что то есть, т.е. в опции есть значение в противном случае false

Функция valOf(v) - если SOME что то вернется к примеру 5, если это NONE то выдаст ошибку. По сути просто мы распаковываем данные.

**17.09.26**

**Пара №4**

Количество операций надо дорожить. И использовать по возможности дешевые операции.

## 1.1 Функция высших порядков

Функции низшего порядка не существуют. Зато есть высшего лол.

Это функция принимает функцию в качестве аргумента. Т.е. для функции аргументом выступает другая функция.

Пример: val find = : int list \* (int -> bool) -> int option

Функции взрывающиеся выражения это функции в префиксной форме.